

Crack the Code of Serverless Design

Chad Green

Senior Systems Architect

Jasper Engines & Transmissions

Level: Intermediate

Session Survey

- Your feedback is very important to us
- Please take a moment to complete the session survey found in the mobile app
- Use the QR code or search for “Converge360 Events” in your app store
- Find this session on the Agenda tab
- Click “Session Evaluation”
- Thank you!



Grew up in software development



Lots of Programming Languages

BASIC

Microsoft BASIC
GW-Basic
QuickBASIC
Visual Basic
VB.NET

C

C
C++
C#
Java

Niche

JavaScript
Pascal
Python
PHP
COBOL

Software development paradigms

Console Applications

Desktop Applications

Distributed Desktop Applications

Web Applications

Distributed Web Applications

COM

DCMO

.NET

Entity Framework

MVC

Service-Oriented Architecture

Microservices

Cloud-Computing

Serverless Computing

Serverless Computing

Functions-as-a-Service

Event-Driven Architecture

Microservices

Work Experience



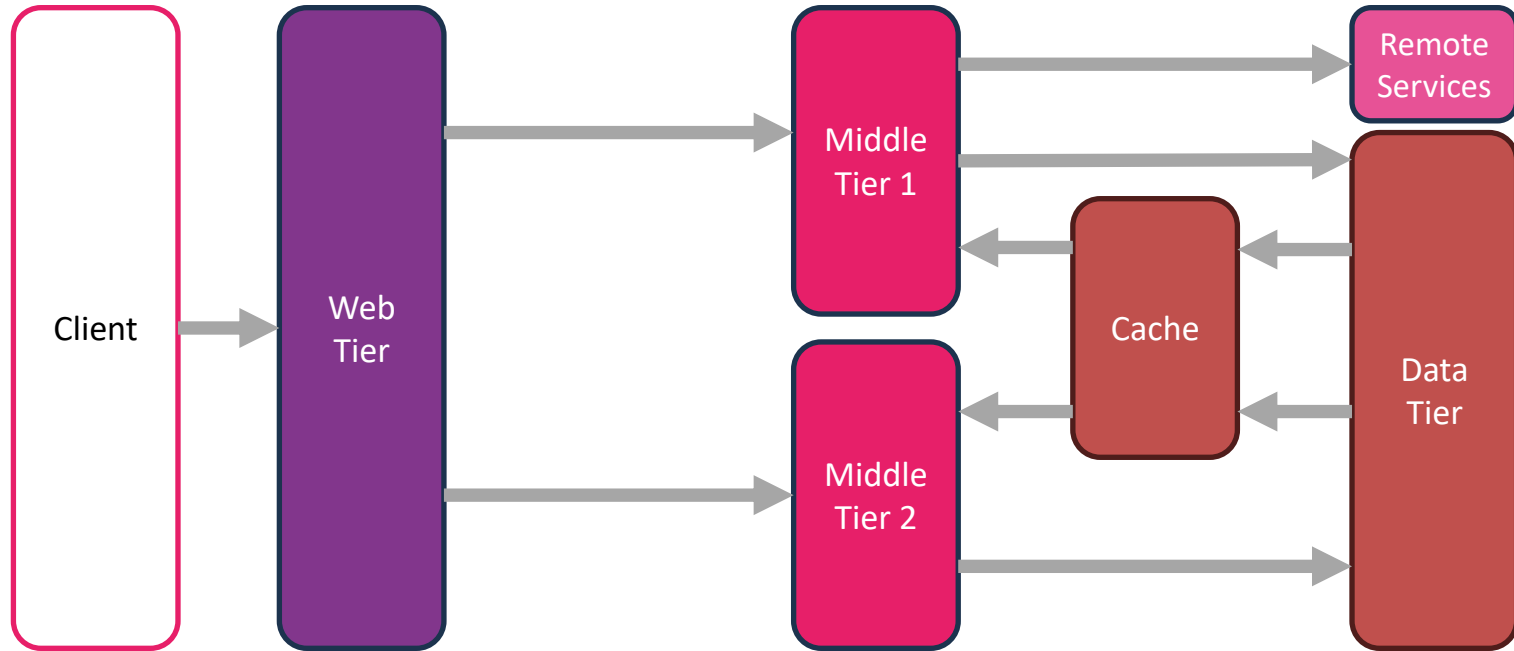
Disclaimer



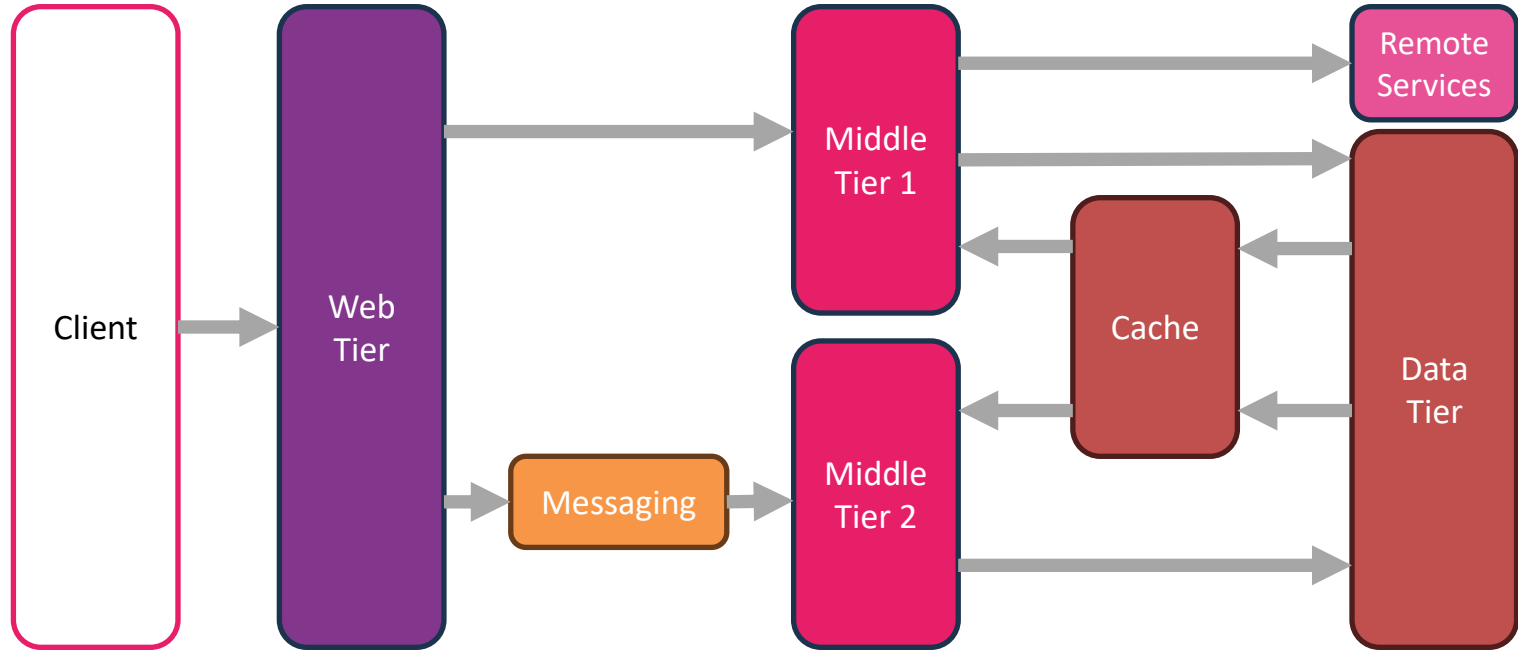
Crack the Code to Scalability

ARCHITECTURE STYLES

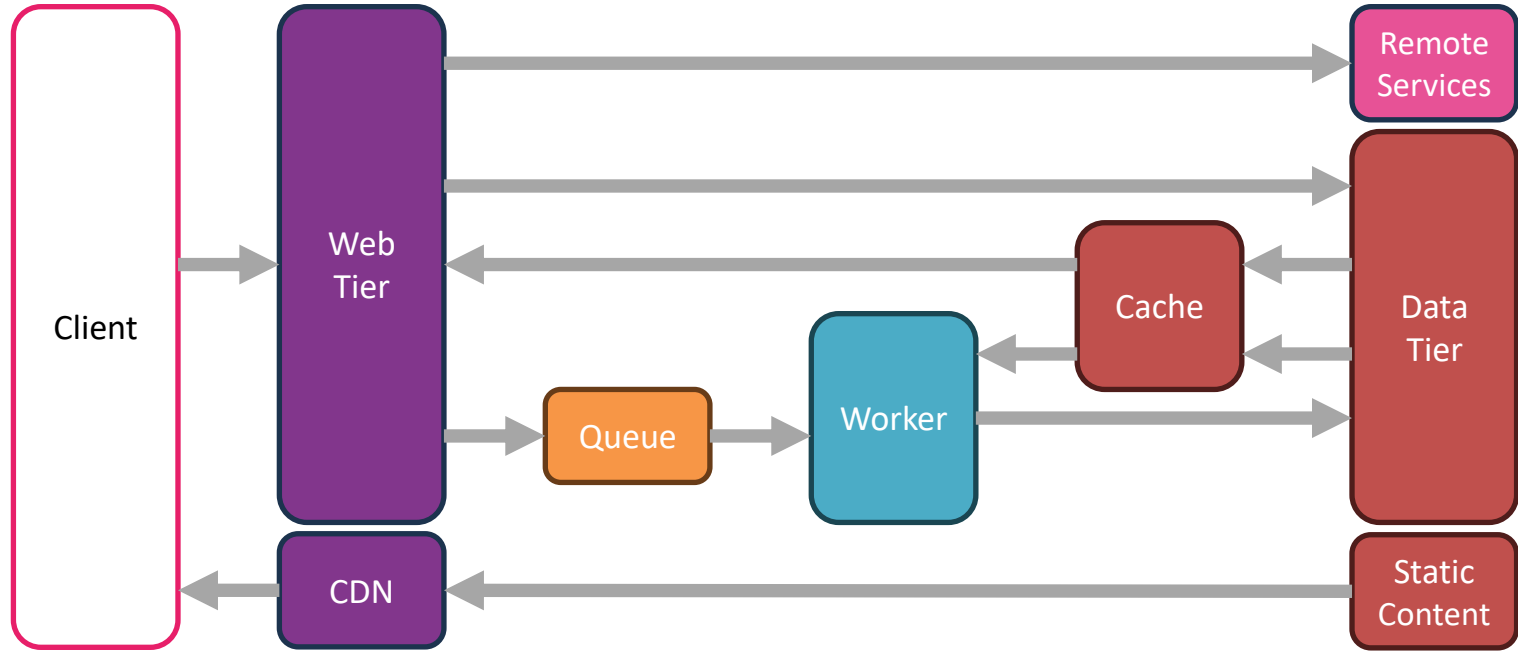
N-Tier



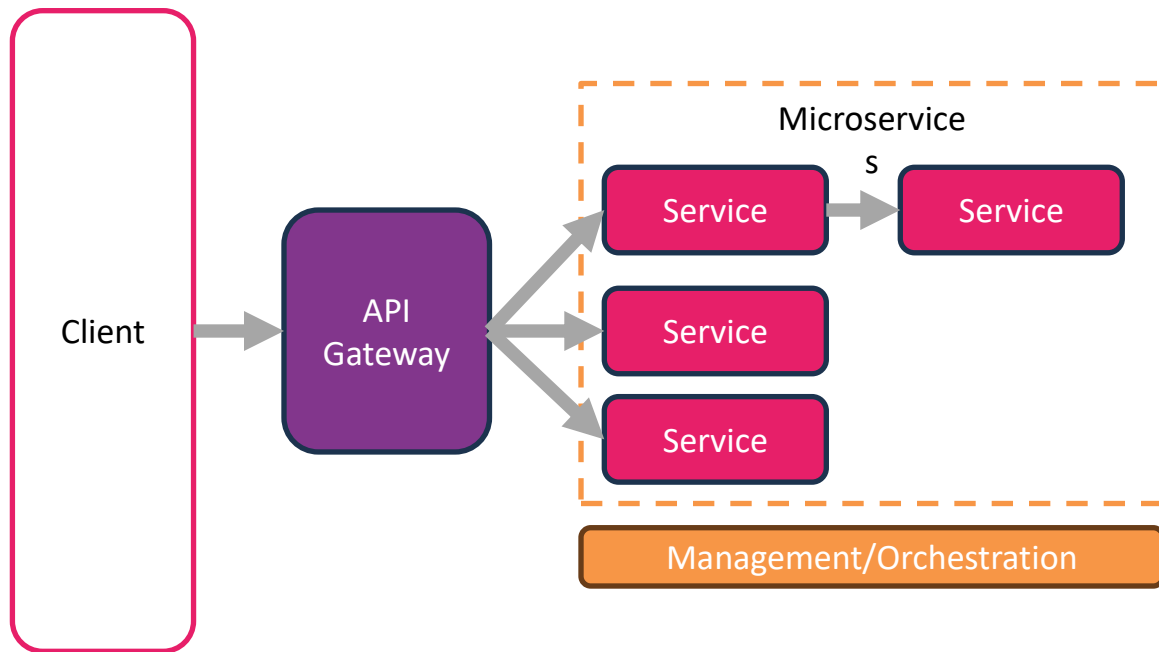
N-Tier with Messaging



Web-Queue-Worker



Microservices



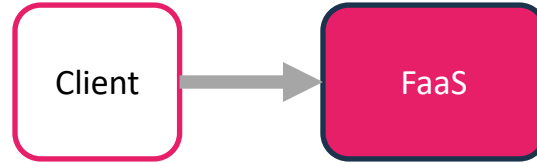
Event-Driven Architecture



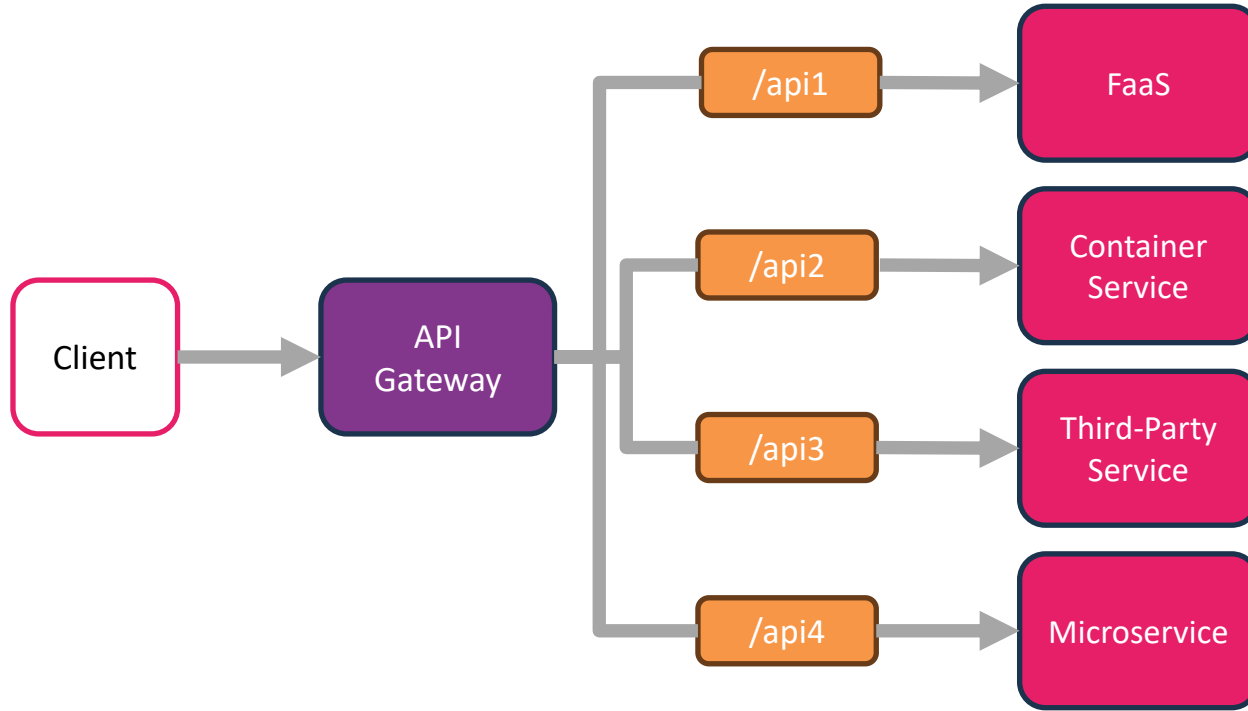
Crack the Code to Scalability

SERVERLESS DESIGN PATTERNS

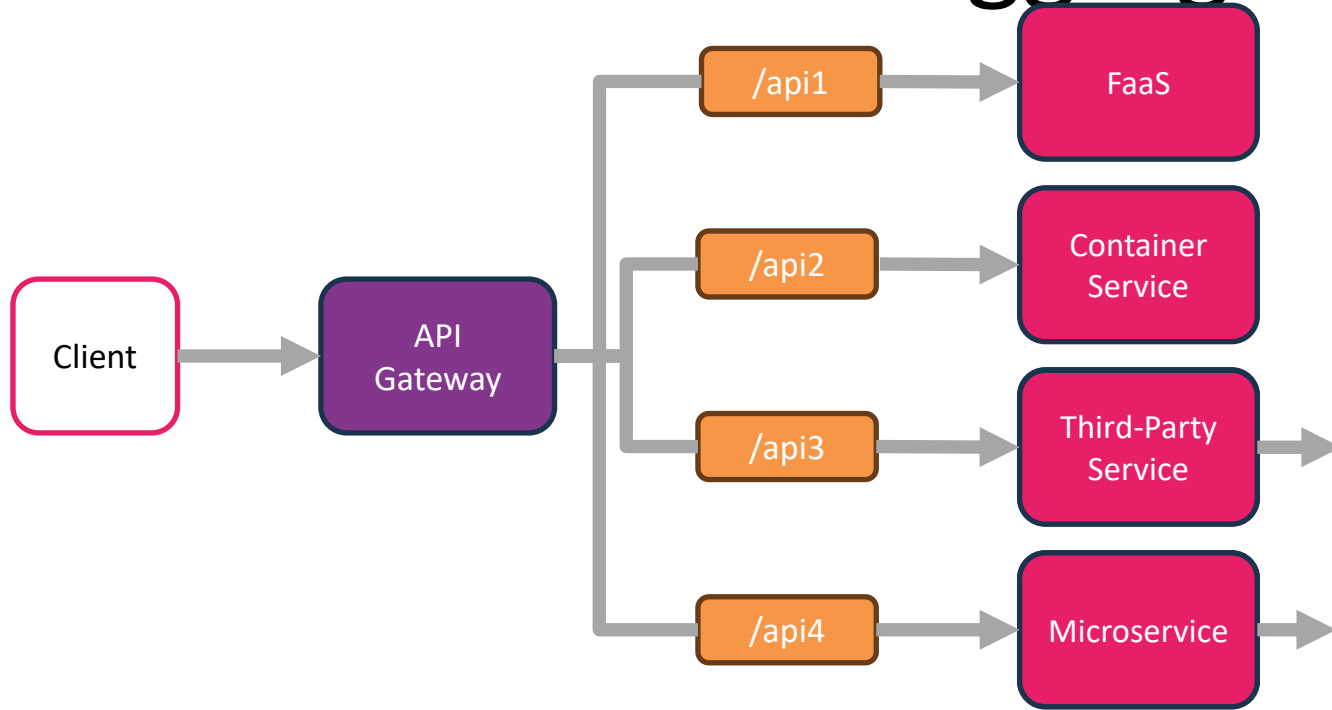
Simple Web Service



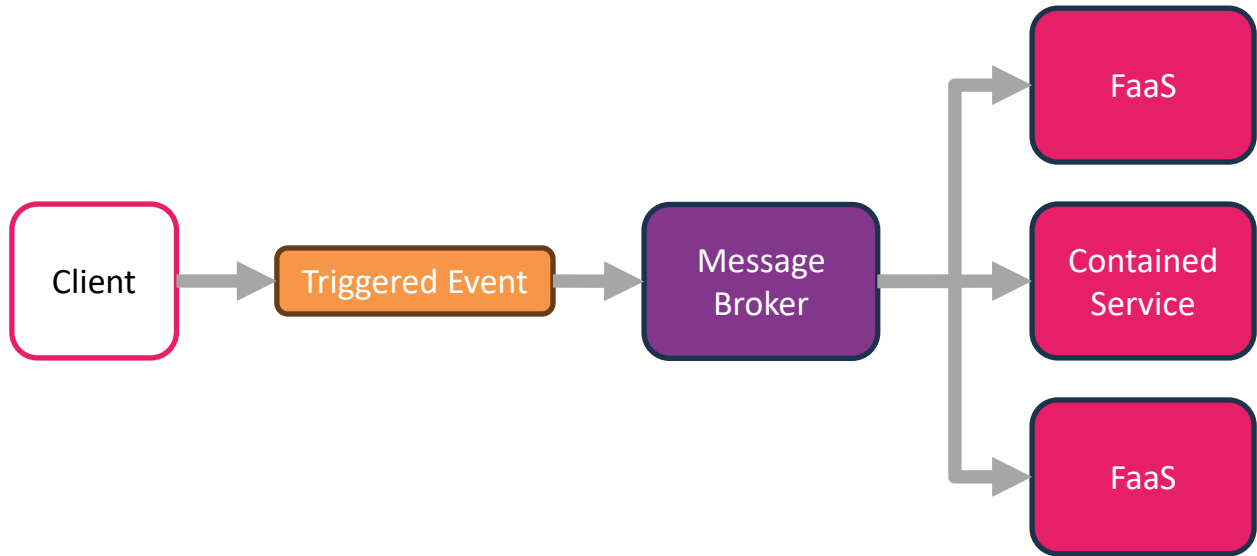
Robust API



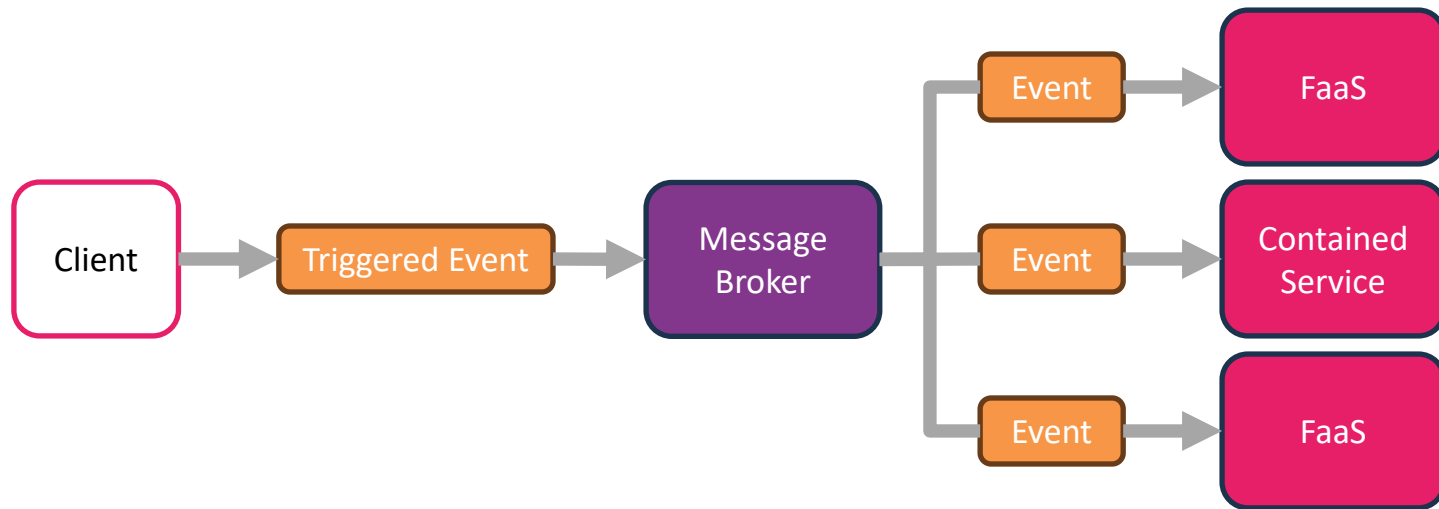
Robust API with Aggregation



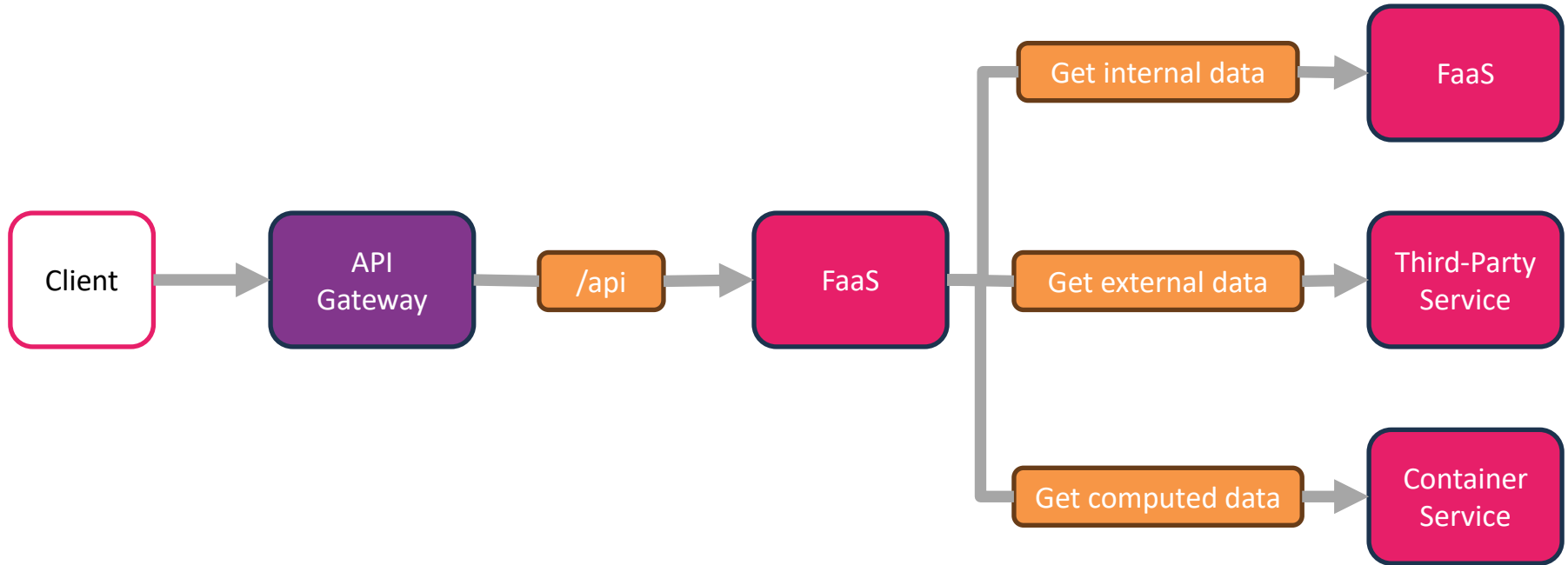
Decoupled Messaging



Publish/Subscribe



Aggregation



Score the Diamond Aggregation Pattern Demo



- Dependencies without being bottlenecked
- Implement resilience patterns

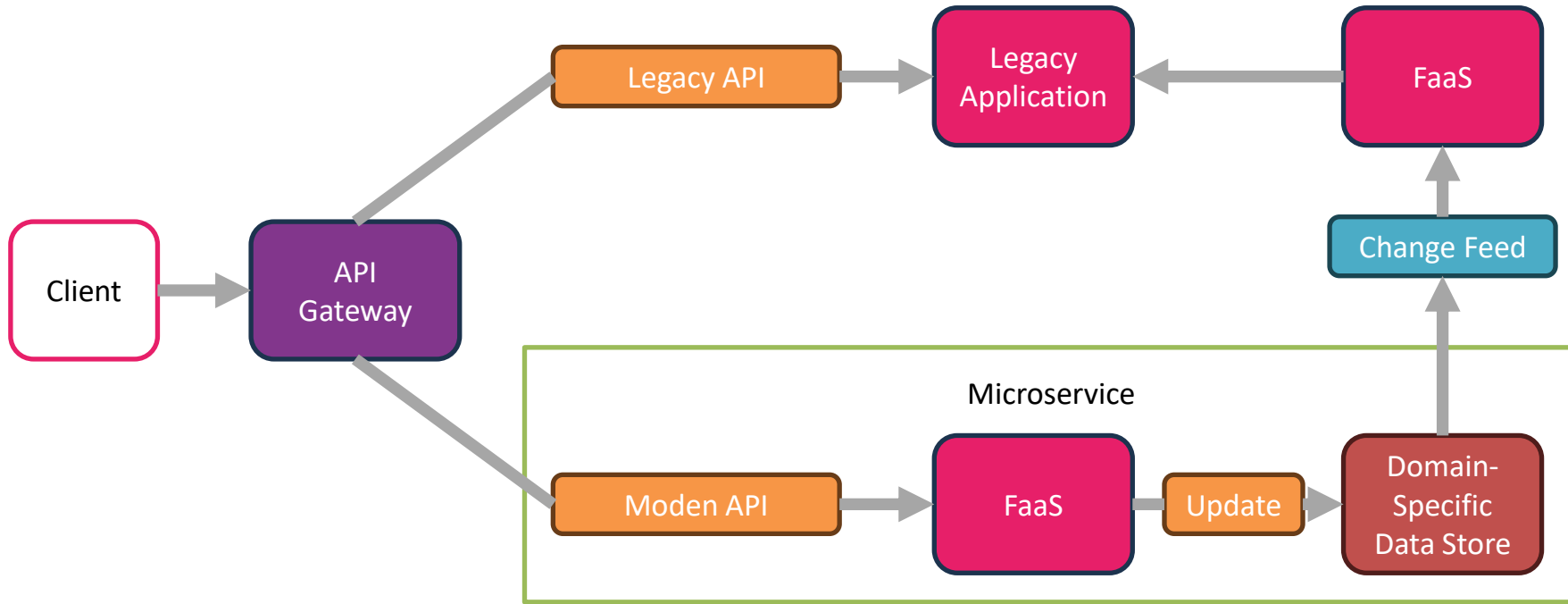
Score the Diamond

Aggregation Pattern Demo

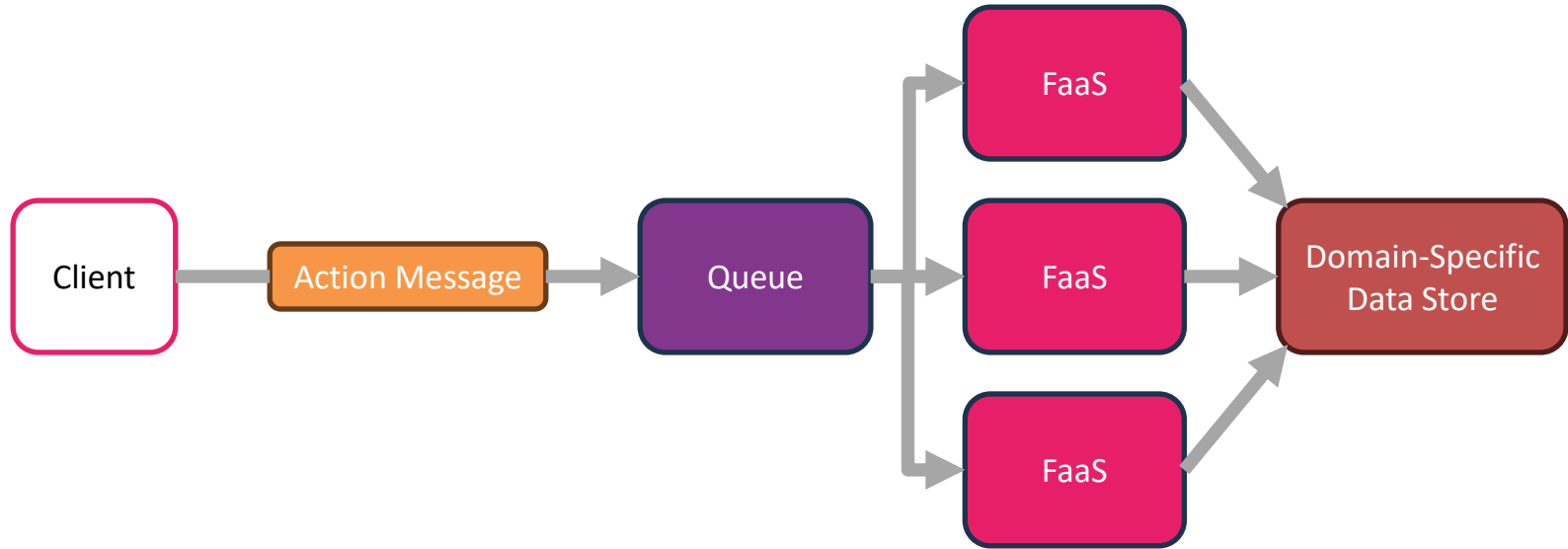


- Simultaneous calls to multiple services
- Timeouts with automatic fallback
- Real-time observation

Strangler



Queue-Based Load Leveling



Pull the Job

Queue-Based Load Leveling Demo



- Absorb burst traffic
- Gracefully handle problems

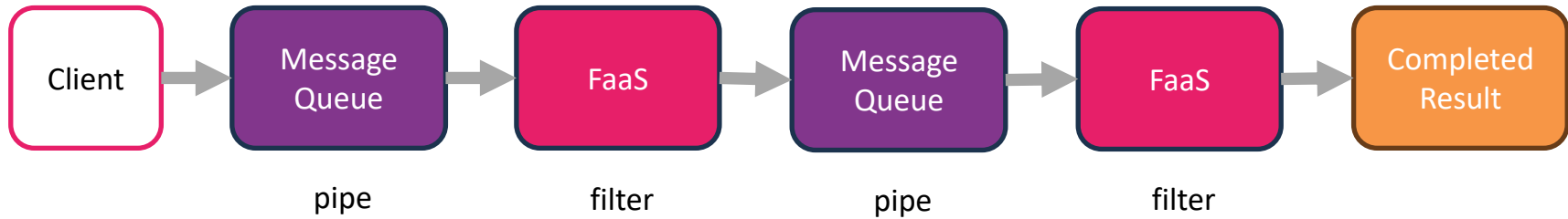
Pull the Job

Queue-Based Load Leveling Demo

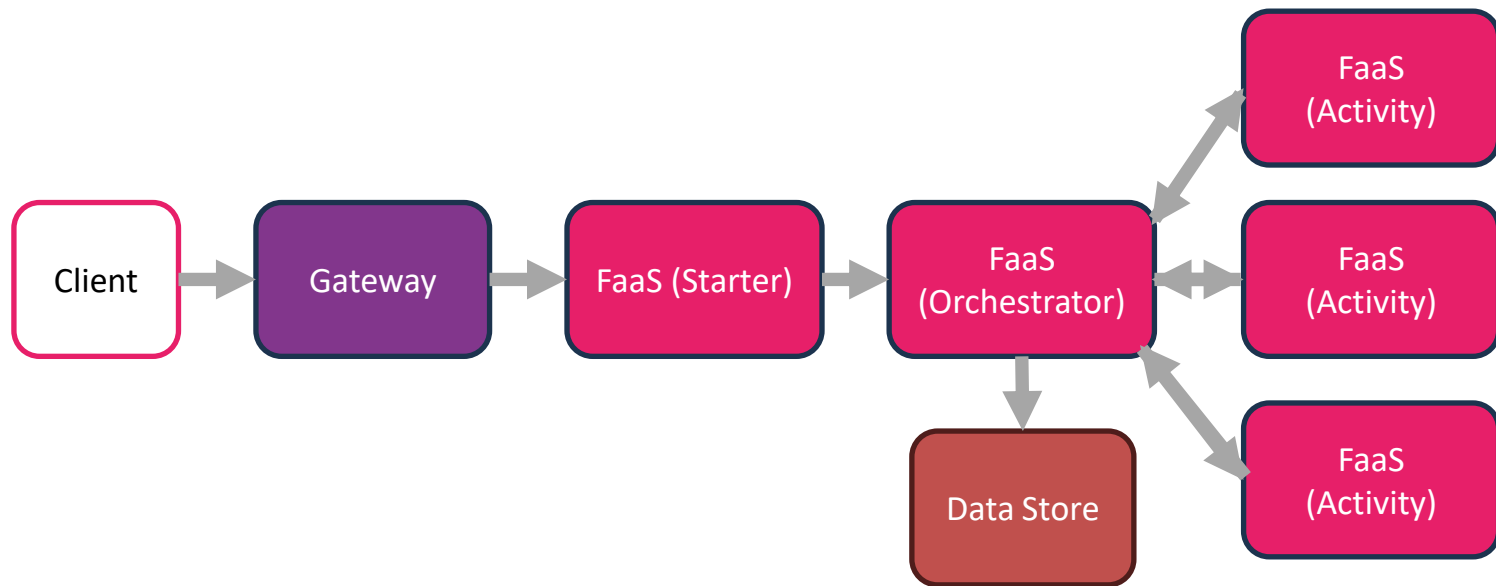


- Accepts burst requests
- Decouples ingestion from processing
- Retry policies
- Real-time queue metrics

Pipes and Filters



Fan-Out/Fan-In



Distribute the Crew

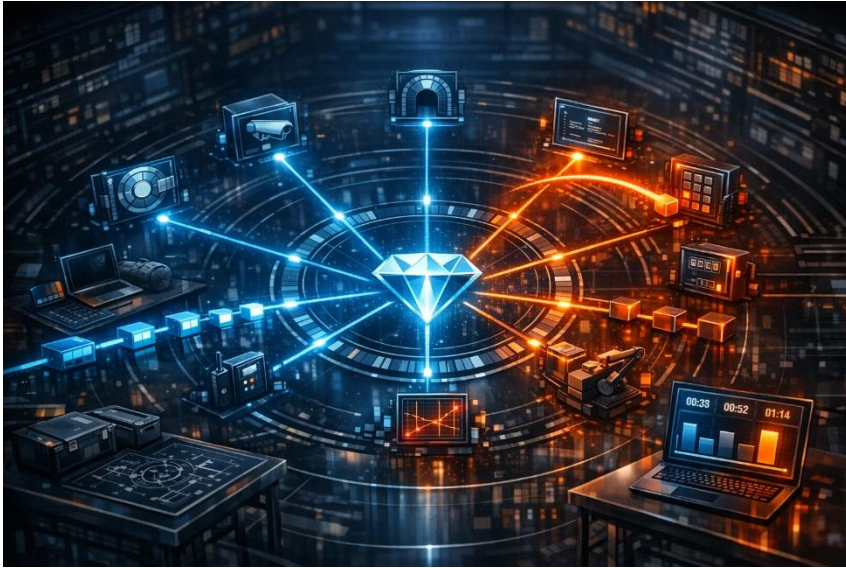
Fan-Out/Fan-In Demo



- Process large workloads in parallel
- Orchestration

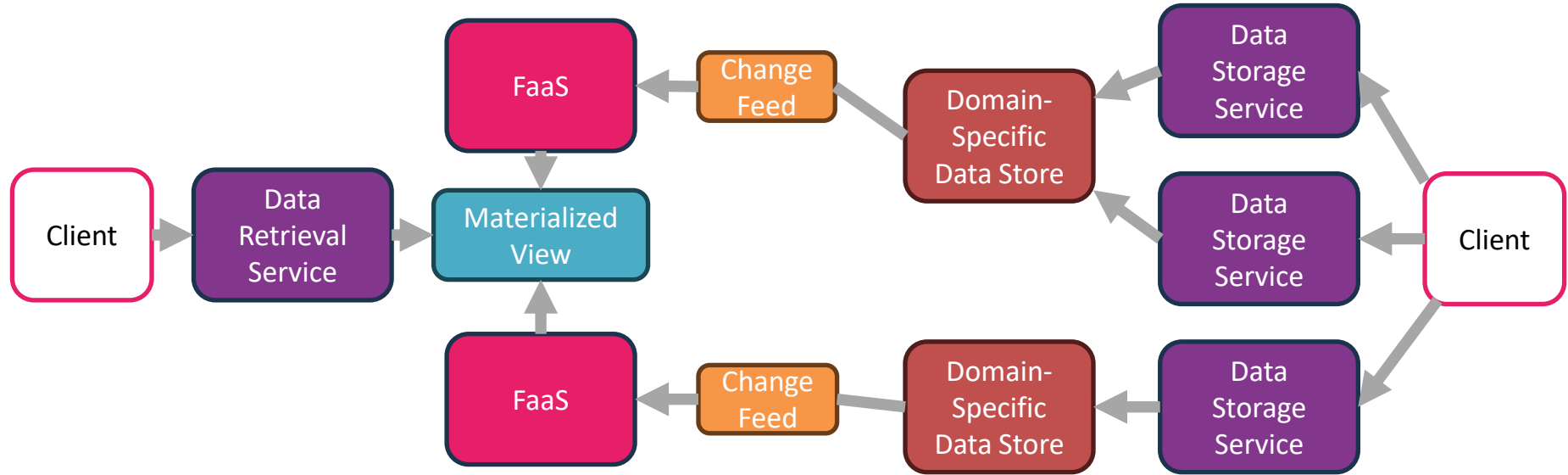
Distribute the Crew

Fan-Out/Fan-In Demo



- Orchestration of parallel activities
- Optimized execution time
- Live monitoring

Materialized Views



Crack the Code to Scalability

PATTERN REALITY CHECK

Pattern Reality Check

Fan-Out/Fan-In

Aggregation

Pub/Sub

Can explode your costs if you are not careful.

Pattern Reality Check

Fan-Out/Fan-In

Aggregation

Pub/Sub

Your API is only as
fast as your slowest
dependency

Pattern Reality Check

Fan-Out/Fan-In

Aggregation

Pub/Sub

Pub/Sub is NOT a
workflow engine.

Crack the Code to Scalability

SERVERLESS BEST PRACTICES

Serverless Best Practices

Stay Focused

Serverless Best Practices

Stay Focused

Handle Errors

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Log and Trace

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Log and Trace

Versioning and
Deployment

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Log and Trace

Versioning and
Deployment

Crack the Code to Scalability

SHORT INTERACTIVE EXERCISE

You need to process 50,000 incoming images, enrich them with metadata, and store them. Which pattern would you choose?

Crack the Code to Scalability

SUMMARY AND KEY TAKEAWAYS

Q&A



Stay Connected

✉ chadgreen@chadgreen.com

💬 TaleLearnCode

🌐 ChadGreen.com

🐦 ChadGreen

in ChadwickEGreen

Please Provide Feedback



<https://github.com/TaleLearnCode/ArchitectLikeABoss>